

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [2]: customers = pd.read_csv("olist_customers_dataset.csv")
geolocation = pd.read_csv("olist_geolocation_dataset.csv")
order_items = pd.read_csv("olist_order_items_dataset.csv")
payments = pd.read_csv("olist_order_payments_dataset.csv")
reviews = pd.read_csv("olist_order_reviews_dataset.csv")
orders = pd.read_csv("olist_orders_dataset.csv")
products = pd.read_csv("olist_products_dataset.csv")
sellers = pd.read_csv("olist_sellers_dataset.csv")
category_translation = pd.read_csv("product_category_name_translation.csv")
```

```
In [3]: print("Customers:", customers.shape)
print("Geolocation:", geolocation.shape)
print("Order Items:", order_items.shape)
print("Payments:", payments.shape)
print("Reviews:", reviews.shape)
print("Orders:", orders.shape)
print("Products:", products.shape)
print("Sellers:", sellers.shape)
print("Category Translation:", category_translation.shape)
```

```
Customers: (99441, 5)
Geolocation: (1000163, 5)
Order Items: (112650, 7)
Payments: (103886, 5)
Reviews: (99224, 7)
Orders: (99441, 8)
Products: (32951, 9)
Sellers: (3095, 4)
Category Translation: (71, 2)
```

```
In [4]: datasets = {
    "Customers": customers,
    "Geolocation": geolocation,
    "Order Items": order_items,
    "Payments": payments,
```

```

    "Reviews": reviews,
    "Orders": orders,
    "Products": products,
    "Sellers": sellers,
    "Category Translation": category_translation
}

```

```

In [5]: for name, df in datasets.items():
        print(f"\n{name}")
        display(df.head())

```

Customers

	customer_id	customer_unique_id	customer_zip_code_prefix	customer_city	customer_state
0	06b8999e2fba1a1fbc88172c00ba8bc7	861eff4711a542e4b93843c6dd7febb0	14409	franca	SP
1	18955e83d337fd6b2def6b18a428ac77	290c77bc529b7ac935b93aa66c333dc3	9790	sao bernardo do campo	SP
2	4e7b3e00288586ebd08712fdd0374a03	060e732b5b29e8181a18229c7b0b2b5e	1151	sao paulo	SP
3	b2b6027bc5c5109e529d4dc6358b12c3	259dac757896d24d7702b9acbbff3f3c	8775	mogi das cruces	SP
4	4f2d8ab171c80ec8364f7c12e35b23ad	345ecd01c38d18a9036ed96c73b8d066	13056	campinas	SP



Geolocation

	geolocation_zip_code_prefix	geolocation_lat	geolocation_lng	geolocation_city	geolocation_state
0	1037	-23.545621	-46.639292	sao paulo	SP
1	1046	-23.546081	-46.644820	sao paulo	SP
2	1046	-23.546129	-46.642951	sao paulo	SP
3	1041	-23.544392	-46.639499	sao paulo	SP
4	1035	-23.541578	-46.641607	sao paulo	SP

Order Items

	order_id	order_item_id	product_id	seller_id	shipping_id
0	00010242fe8c5a6d1ba2dd792cb16214	1	4244733e06e7ecb4970a6e2683c13e61	48436dade18ac8b2bce089ec2a041202	201
1	00018f77f2f0320c557190d7a144bdd3	1	e5f2d52b802189ee658865ca93d83a8f	dd7ddc04e1b6c2c614352b383efe2d36	201
2	000229ec398224ef6ca0657da4fc703e	1	c777355d18b72b67abbef9df44fd0fd	5b51032eddd242adc84c38acab88f23d	201
3	00024acbcd0a6daa1e931b038114c75	1	7634da152a4610f1595efa32f14722fc	9d7a1d34a5052409006425275ba1c2b4	201
4	00042b26cf59d7ce69dfabb4e55b4fd9	1	ac6c3623068f30de03045865e4e10089	df560393f3a51e74553ab94004ba5c87	201



Payments

	order_id	payment_sequential	payment_type	payment_installments	payment_value
0	b81ef226f3fe1789b1e8b2acac839d17	1	credit_card	8	99.33
1	a9810da82917af2d9aefd1278f1dcfa0	1	credit_card	1	24.39
2	25e8ea4e93396b6fa0d3dd708e76c1bd	1	credit_card	1	65.71
3	ba78997921bbcdc1373bb41e913ab953	1	credit_card	8	107.78
4	42fdf880ba16b47b59251dd489d4441a	1	credit_card	2	128.45

Reviews

	review_id	order_id	review_score	review_comment_title	review_comment_r
0	7bc2406110b926393aa56f80a40eba40	73fc7af87114b39712e6da79b0a377eb	4	NaN	
1	80e641a11e56f04c1ad469d5645fdfde	a548910a1c6147796b98fdf73dbeba33	5	NaN	
2	228ce5500dc1d8e020d8d1322874b6f0	f9e4b658b201a9f2ecdecbb34bed034b	5	NaN	
3	e64fb393e7b32834bb789ff8bb30750e	658677c97b385a9be170737859d3511b	5	NaN	Recebi bem antes de
4	f7c4243c7fe1938f181bec41a392bdeb	8e6bfb81e283fa7e4f11123a3fb894f1	5	NaN	Parabéns lojas adorei compra



Orders

	order_id	customer_id	order_status	order_purchase_timestamp	order_approve
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02 10:56:33	2017-11-01 11:0
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-07-24 20:41:37	2018-07-24 03:2
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-08-08 08:38:49	2018-08-08 08:5
3	949d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcdbec7375364d82	delivered	2017-11-18 19:28:06	2017-11-18 19:4
4	ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c	delivered	2018-02-13 21:18:39	2018-02-13 22:2



Products

	product_id	product_category_name	product_name_lenght	product_description_lenght	product_photos_lenght
0	1e9e8ef04dbcff4541ed26657ea517e5	perfumaria	40.0	287.0	
1	3aa071139cb16b67ca9e5dea641aaa2f	artes	44.0	276.0	
2	96bd76ec8810374ed1b65e291975717f	esporte_lazer	46.0	250.0	
3	cef67bcfe19066a932b7673e239eb23d	bebes	27.0	261.0	
4	9dc1a7de274444849c219cff195d0b71	utilidades_domesticas	37.0	402.0	



Sellers

	seller_id	seller_zip_code_prefix	seller_city	seller_state
0	3442f8959a84dea7ee197c632cb2df15	13023	campinas	SP
1	d1b65fc7debc3361ea86b5f14c68d2e2	13844	mogi guacu	SP
2	ce3ad9de960102d0677a81f5d0bb7b2d	20031	rio de janeiro	RJ
3	c0f3eea2e14555b6faeea3dd58c1b1c3	4195	sao paulo	SP
4	51a04a8a6bdbcb23deccc82b0b80742cf	12914	braganca paulista	SP

Category Translation

	product_category_name	product_category_name_english
0	beleza_saude	health_beauty
1	informatica_acessorios	computers_accessories
2	automotivo	auto
3	cama_mesa_banho	bed_bath_table
4	moveis_decoracao	furniture_decor

```
In [6]: for name, df in datasets.items():
        print(f"\n{name}")
        df.info()
```

Customers

<class 'pandas.DataFrame'>

RangeIndex: 99441 entries, 0 to 99440

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	customer_id	99441 non-null	str
1	customer_unique_id	99441 non-null	str
2	customer_zip_code_prefix	99441 non-null	int64
3	customer_city	99441 non-null	str
4	customer_state	99441 non-null	str

dtypes: int64(1), str(4)

memory usage: 11.0 MB

Geolocation

<class 'pandas.DataFrame'>

RangeIndex: 1000163 entries, 0 to 1000162

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	geolocation_zip_code_prefix	1000163 non-null	int64
1	geolocation_lat	1000163 non-null	float64
2	geolocation_lng	1000163 non-null	float64
3	geolocation_city	1000163 non-null	str
4	geolocation_state	1000163 non-null	str

dtypes: float64(2), int64(1), str(2)

memory usage: 50.1 MB

Order Items

<class 'pandas.DataFrame'>

RangeIndex: 112650 entries, 0 to 112649

Data columns (total 7 columns):

#	Column	Non-Null Count	Dtype
0	order_id	112650 non-null	str
1	order_item_id	112650 non-null	int64
2	product_id	112650 non-null	str
3	seller_id	112650 non-null	str
4	shipping_limit_date	112650 non-null	str
5	price	112650 non-null	float64
6	freight_value	112650 non-null	float64

dtypes: float64(2), int64(1), str(4)

memory usage: 18.4 MB

Payments

<class 'pandas.DataFrame'>

RangeIndex: 103886 entries, 0 to 103885

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	order_id	103886 non-null	str
1	payment_sequential	103886 non-null	int64
2	payment_type	103886 non-null	str
3	payment_installments	103886 non-null	int64
4	payment_value	103886 non-null	float64

dtypes: float64(1), int64(2), str(2)

memory usage: 8.1 MB

Reviews

<class 'pandas.DataFrame'>

RangeIndex: 99224 entries, 0 to 99223

Data columns (total 7 columns):

#	Column	Non-Null Count	Dtype
0	review_id	99224 non-null	str
1	order_id	99224 non-null	str
2	review_score	99224 non-null	int64
3	review_comment_title	11568 non-null	str
4	review_comment_message	40977 non-null	str
5	review_creation_date	99224 non-null	str
6	review_answer_timestamp	99224 non-null	str

dtypes: int64(1), str(6)

memory usage: 17.8 MB

Orders

<class 'pandas.DataFrame'>

RangeIndex: 99441 entries, 0 to 99440

Data columns (total 8 columns):

#	Column	Non-Null Count	Dtype
0	order_id	99441 non-null	str
1	customer_id	99441 non-null	str
2	order_status	99441 non-null	str
3	order_purchase_timestamp	99441 non-null	str

```
4  order_approved_at          99281 non-null  str
5  order_delivered_carrier_date 97658 non-null  str
6  order_delivered_customer_date 96476 non-null  str
7  order_estimated_delivery_date 99441 non-null  str
dtypes: str(8)
memory usage: 21.9 MB
```

Products

```
<class 'pandas.DataFrame'>
RangeIndex: 32951 entries, 0 to 32950
Data columns (total 9 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   product_id                           32951 non-null  str
1   product_category_name                 32341 non-null  str
2   product_name_lenght                   32341 non-null  float64
3   product_description_lenght            32341 non-null  float64
4   product_photos_qty                   32341 non-null  float64
5   product_weight_g                      32949 non-null  float64
6   product_length_cm                     32949 non-null  float64
7   product_height_cm                     32949 non-null  float64
8   product_width_cm                      32949 non-null  float64
dtypes: float64(7), str(2)
memory usage: 3.7 MB
```

Sellers

```
<class 'pandas.DataFrame'>
RangeIndex: 3095 entries, 0 to 3094
Data columns (total 4 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   seller_id                             3095 non-null  str
1   seller_zip_code_prefix                3095 non-null  int64
2   seller_city                           3095 non-null  str
3   seller_state                          3095 non-null  str
dtypes: int64(1), str(3)
memory usage: 230.3 KB
```

Category Translation

```
<class 'pandas.DataFrame'>
RangeIndex: 71 entries, 0 to 70
Data columns (total 2 columns):
```


#	Column	Non-Null Count	Dtype
0	product_category_name	71 non-null	str
1	product_category_name_english	71 non-null	str

dtypes: str(2)
memory usage: 3.5 KB

```
In [7]: print("Orders missing values: \n", orders.isnull().sum())
print("\nOrder Items missing values: \n", order_items.isnull().sum())
print("\nCustomers missing values: \n", customers.isnull().sum())
print("\nGeolocation missing values: \n", geolocation.isnull().sum())
print("\nPayments missing values: \n", payments.isnull().sum())
print("\nReviews missing values: \n", reviews.isnull().sum())
print("\nProducts missing values: \n", products.isnull().sum())
print("\nSellers missing values: \n", sellers.isnull().sum())
print("\nCategory Translation missing values: \n", category_translation.isnull().sum())
```

Orders missing values:

order_id	0
customer_id	0
order_status	0
order_purchase_timestamp	0
order_approved_at	160
order_delivered_carrier_date	1783
order_delivered_customer_date	2965
order_estimated_delivery_date	0

dtype: int64

Order Items missing values:

order_id	0
order_item_id	0
product_id	0
seller_id	0
shipping_limit_date	0
price	0
freight_value	0

dtype: int64

Customers missing values:

customer_id	0
customer_unique_id	0
customer_zip_code_prefix	0
customer_city	0
customer_state	0

dtype: int64

Geolocation missing values:

geolocation_zip_code_prefix	0
geolocation_lat	0
geolocation_lng	0
geolocation_city	0
geolocation_state	0

dtype: int64

Payments missing values:

order_id	0
payment_sequential	0
payment_type	0
payment_installments	0

```
payment_value      0
dtype: int64
```

Reviews missing values:

```
review_id          0
order_id           0
review_score        0
review_comment_title 87656
review_comment_message 58247
review_creation_date 0
review_answer_timestamp 0
dtype: int64
```

Products missing values:

```
product_id         0
product_category_name 610
product_name_lenght 610
product_description_lenght 610
product_photos_qty 610
product_weight_g    2
product_length_cm   2
product_height_cm    2
product_width_cm     2
dtype: int64
```

Sellers missing values:

```
seller_id          0
seller_zip_code_prefix 0
seller_city         0
seller_state        0
dtype: int64
```

Category Translation missing values:

```
product_category_name 0
product_category_name_english 0
dtype: int64
```

```
In [8]: datasets = {
    "Customers": customers,
    "Geolocation": geolocation,
    "Order Items": order_items,
    "Payments": payments,
```

```
    "Reviews":reviews,  
    "Orders":orders,  
    "Products":products,  
    "Sellers":sellers,  
    "Category Translation":  
    category_translation  
}  
for name, df in datasets.items():  
    missing = df.isnull().sum()  
    missing = missing[missing > 0]  
  
    print(f"\n{name}")  
    print(missing)
```

```
Customers
Series([], dtype: int64)

Geolocation
Series([], dtype: int64)

Order Items
Series([], dtype: int64)

Payments
Series([], dtype: int64)

Reviews
review_comment_title      87656
review_comment_message    58247
dtype: int64

Orders
order_approved_at         160
order_delivered_carrier_date 1783
order_delivered_customer_date 2965
dtype: int64

Products
product_category_name      610
product_name_lenght        610
product_description_lenght  610
product_photos_qty         610
product_weight_g           2
product_length_cm          2
product_height_cm          2
product_width_cm           2
dtype: int64

Sellers
Series([], dtype: int64)

Category Translation
Series([], dtype: int64)
```

```
In [9]: for name, df in datasets.items():
        missing = df.isnull().sum()
```

```

missing_percent = (missing / len(df)) * 100

result = pd.DataFrame({'Missing Count' : missing, 'Missing %': missing_percent})
result = result[result['Missing Count'] > 0]

if not result.empty:
    print(f"\n{name}")
    print(result.round(2))

```

Reviews

	Missing Count	Missing %
review_comment_title	87656	88.34
review_comment_message	58247	58.70

Orders

	Missing Count	Missing %
order_approved_at	160	0.16
order_delivered_carrier_date	1783	1.79
order_delivered_customer_date	2965	2.98

Products

	Missing Count	Missing %
product_category_name	610	1.85
product_name_lenght	610	1.85
product_description_lenght	610	1.85
product_photos_qty	610	1.85
product_weight_g	2	0.01
product_length_cm	2	0.01
product_height_cm	2	0.01
product_width_cm	2	0.01

```

In [10]: for name, df in datasets.items():
          duplicates = df.duplicated().sum()

          print(f"\n{name}")
          print("Duplicate Rows:",duplicates)

```

Customers
Duplicate Rows: 0

Geolocation
Duplicate Rows: 261831

Order Items
Duplicate Rows: 0

Payments
Duplicate Rows: 0

Reviews
Duplicate Rows: 0

Orders
Duplicate Rows: 0

Products
Duplicate Rows: 0

Sellers
Duplicate Rows: 0

Category Translation
Duplicate Rows: 0

```
In [11]: for name, df in datasets.items():  
          print(f"\n{name}")  
          display(df.describe())
```

Customers

customer_zip_code_prefix	
count	99441.000000
mean	35137.474583
std	29797.938996
min	1003.000000
25%	11347.000000
50%	24416.000000
75%	58900.000000
max	99990.000000

Geolocation

	geolocation_zip_code_prefix	geolocation_lat	geolocation_lng
count	1.000163e+06	1.000163e+06	1.000163e+06
mean	3.657417e+04	-2.117615e+01	-4.639054e+01
std	3.054934e+04	5.715866e+00	4.269748e+00
min	1.001000e+03	-3.660537e+01	-1.014668e+02
25%	1.107500e+04	-2.360355e+01	-4.857317e+01
50%	2.653000e+04	-2.291938e+01	-4.663788e+01
75%	6.350400e+04	-1.997962e+01	-4.376771e+01
max	9.999000e+04	4.506593e+01	1.211054e+02

Order Items

	order_item_id	price	freight_value
count	112650.000000	112650.000000	112650.000000
mean	1.197834	120.653739	19.990320
std	0.705124	183.633928	15.806405
min	1.000000	0.850000	0.000000
25%	1.000000	39.900000	13.080000
50%	1.000000	74.990000	16.260000
75%	1.000000	134.900000	21.150000
max	21.000000	6735.000000	409.680000

Payments

	payment_sequential	payment_installments	payment_value
count	103886.000000	103886.000000	103886.000000
mean	1.092679	2.853349	154.100380
std	0.706584	2.687051	217.494064
min	1.000000	0.000000	0.000000
25%	1.000000	1.000000	56.790000
50%	1.000000	1.000000	100.000000
75%	1.000000	4.000000	171.837500
max	29.000000	24.000000	13664.080000

Reviews

review_score	
count	99224.000000
mean	4.086421
std	1.347579
min	1.000000
25%	4.000000
50%	5.000000
75%	5.000000
max	5.000000

Orders

	order_id	customer_id	order_status	order_purchase_timestamp	order_app
count	99441	99441	99441	99441	
unique	99441	99441	8	98875	
top	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2018-03-31 15:08:21	20
freq	1	1	96478	3	



Products

	product_name_lenght	product_description_lenght	product_photos_qty	product_weight_g	product_length_cm	product_heigl
count	32341.000000	32341.000000	32341.000000	32949.000000	32949.000000	32949.000000
mean	48.476949	771.495285	2.188986	2276.472488	30.815078	16.914458
std	10.245741	635.115225	1.736766	4282.038731	16.914458	13.614458
min	5.000000	4.000000	1.000000	0.000000	7.000000	2.000000
25%	42.000000	339.000000	1.000000	300.000000	18.000000	8.000000
50%	51.000000	595.000000	1.000000	700.000000	25.000000	13.000000
75%	57.000000	972.000000	3.000000	1900.000000	38.000000	21.000000
max	76.000000	3992.000000	20.000000	40425.000000	105.000000	105.000000



Sellers

	seller_zip_code_prefix
count	3095.000000
mean	32291.059451
std	32713.453830
min	1001.000000
25%	7093.500000
50%	14940.000000
75%	64552.500000
max	99730.000000

Category Translation

	product_category_name	product_category_name_english
count	71	71
unique	71	71
top	beleza_saude	health_beauty
freq	1	1

```
In [12]: for name, df in datasets.items():
          print(f"\n{'='*50}")
          print(name)
          print("="*50)
          print(df.dtypes)
```

```
=====
Customers
=====
customer_id          str
customer_unique_id   str
customer_zip_code_prefix  int64
customer_city        str
customer_state        str
dtype: object
```

```
=====
Geolocation
=====
geolocation_zip_code_prefix  int64
geolocation_lat              float64
geolocation_lng              float64
geolocation_city             str
geolocation_state            str
dtype: object
```

```
=====
Order Items
=====
order_id          str
order_item_id     int64
product_id        str
seller_id         str
shipping_limit_date  str
price             float64
freight_value      float64
dtype: object
```

```
=====
Payments
=====
order_id          str
payment_sequential  int64
payment_type       str
payment_installments  int64
payment_value       float64
dtype: object
```

```

=====
Reviews
=====
review_id          str
order_id           str
review_score        int64
review_comment_title str
review_comment_message str
review_creation_date str
review_answer_timestamp str
dtype: object

=====
Orders
=====
order_id          str
customer_id        str
order_status       str
order_purchase_timestamp str
order_approved_at str
order_delivered_carrier_date str
order_delivered_customer_date str
order_estimated_delivery_date str
dtype: object

=====
Products
=====
product_id          str
product_category_name str
product_name_lenght float64
product_description_lenght float64
product_photos_qty float64
product_weight_g float64
product_length_cm float64
product_height_cm float64
product_width_cm float64
dtype: object

=====
Sellers
=====

```

```
seller_id          str
seller_zip_code_prefix  int64
seller_city        str
seller_state       str
dtype: object
```

```
=====
Category Translation
=====
product_category_name      str
product_category_name_english  str
dtype: object
```

```
In [13]: customers["customer_zip_code_prefix"] = customers["customer_zip_code_prefix"].astype(str)
         geolocation["geolocation_zip_code_prefix"] = geolocation["geolocation_zip_code_prefix"].astype(str)
         sellers["seller_zip_code_prefix"] = sellers["seller_zip_code_prefix"].astype(str)
```

```
In [14]: print(customers["customer_zip_code_prefix"].dtype)
         print(geolocation["geolocation_zip_code_prefix"].dtype)
         print(sellers["seller_zip_code_prefix"].dtype)
```

```
str
str
str
```

```
In [15]: order_items["shipping_limit_date"] = pd.to_datetime(
         order_items["shipping_limit_date"],
         errors="coerce"
         )
```

```
In [16]: order_items.dtypes
```

```
Out[16]: order_id          str
         order_item_id      int64
         product_id         str
         seller_id          str
         shipping_limit_date  datetime64[us]
         price              float64
         freight_value      float64
         dtype: object
```

```
In [17]: reviews["review_creation_date"] = pd.to_datetime(reviews["review_creation_date"], errors = "coerce")
reviews["review_answer_timestamp"] = pd.to_datetime(reviews["review_answer_timestamp"], errors="coerce")
```

```
In [18]: orders["order_purchase_timestamp"] = pd.to_datetime(orders["order_purchase_timestamp"], errors="coerce")
orders["order_approved_at"] = pd.to_datetime(orders["order_approved_at"], errors="coerce")
orders["order_delivered_carrier_date"] = pd.to_datetime(orders["order_delivered_carrier_date"], errors="coerce")
orders["order_delivered_customer_date"] = pd.to_datetime(orders["order_delivered_customer_date"], errors="coerce")
orders["order_estimated_delivery_date"] = pd.to_datetime(orders["order_estimated_delivery_date"], errors="coerce")
```

```
In [19]: orders.dtypes
```

```
Out[19]: order_id                str
customer_id                str
order_status                str
order_purchase_timestamp    datetime64[us]
order_approved_at           datetime64[us]
order_delivered_carrier_date datetime64[us]
order_delivered_customer_date datetime64[us]
order_estimated_delivery_date datetime64[us]
dtype: object
```

```
In [20]: for name, df in datasets.items():
missing = df.isnull().sum()
missing_percent = (missing / len(df)) * 100

result = pd.DataFrame({
    "Missing Count": missing,
    "Missing %": missing_percent
})

result = result[result["Missing Count"] > 0]

if not result.empty:
    print(f"\n{name}")
    print(result.round(2))
```


Reviews

	Missing Count	Missing %
review_comment_title	87656	88.34
review_comment_message	58247	58.70

Orders

	Missing Count	Missing %
order_approved_at	160	0.16
order_delivered_carrier_date	1783	1.79
order_delivered_customer_date	2965	2.98

Products

	Missing Count	Missing %
product_category_name	610	1.85
product_name_lenght	610	1.85
product_description_lenght	610	1.85
product_photos_qty	610	1.85
product_weight_g	2	0.01
product_length_cm	2	0.01
product_height_cm	2	0.01
product_width_cm	2	0.01

```
In [21]: product_num_cols = [  
        "product_weight_g",  
        "product_length_cm",  
        "product_height_cm",  
        "product_width_cm"  
    ]  
  
    for col in product_num_cols:  
        products[col] = products[col].fillna(products[col].median())
```

```
In [22]: products[product_num_cols].isnull().sum()
```

```
Out[22]: product_weight_g      0  
product_length_cm      0  
product_height_cm      0  
product_width_cm      0  
dtype: int64
```

```
In [23]: product_num_cols = [  
        "product_name_lenght",
```

```

        "product_description_lenght",
        "product_photos_qty"
    ]
    for col in product_num_cols: products[col] = products[col].fillna(products[col].median())

```

```
In [24]: products["product_category_name"] = products["product_category_name"].fillna("Unknown")
```

```
In [25]: products.isnull().sum()[products.isnull().sum() > 0]
```

```
Out[25]: Series([], dtype: int64)
```

```
In [26]: for name, df in datasets.items():
          print(name, ":", df.isnull().sum().sum())
```

```

Customers : 0
Geolocation : 0
Order Items : 0
Payments : 0
Reviews : 145903
Orders : 4908
Products : 0
Sellers : 0
Category Translation : 0

```

```
In [27]: reviews["review_comment_title"] = reviews["review_comment_title"].fillna("No Comment")
         reviews["review_comment_message"] = reviews["review_comment_message"].fillna("No Comment")
```

```
In [28]: reviews.isnull().sum()
```

```

Out[28]: review_id          0
         order_id          0
         review_score       0
         review_comment_title  0
         review_comment_message  0
         review_creation_date  0
         review_answer_timestamp  0
         dtype: int64

```

```
In [29]: orders["orders_approved_at"] = pd.to_datetime(orders["order_approved_at"], errors="coerce")
```

```
In [30]: orders["order_delivered_carrier_date"] = pd.to_datetime(orders["order_delivered_carrier_date"],errors = "coerce")
orders["order_delivered_customer_date"] = pd.to_datetime(orders["order_delivered_customer_date"],errors="coerce")
```

```
In [31]: orders[["order_delivered_carrier_date","order_delivered_customer_date"]].dtypes
```

```
Out[31]: order_delivered_carrier_date    datetime64[us]
order_delivered_customer_date    datetime64[us]
dtype: object
```

```
In [32]: orders.isnull().sum()
```

```
Out[32]: order_id                0
customer_id                0
order_status                0
order_purchase_timestamp    0
order_approved_at          160
order_delivered_carrier_date 1783
order_delivered_customer_date 2965
order_estimated_delivery_date    0
orders_approved_at          160
dtype: int64
```

```
In [33]: for name, df in datasets.items():
print(name,"duplicates:",df.duplicated().sum())
```

```
Customers duplicates: 0
Geolocation duplicates: 261831
Order Items duplicates: 0
Payments duplicates: 0
Reviews duplicates: 0
Orders duplicates: 0
Products duplicates: 0
Sellers duplicates: 0
Category Translation duplicates: 0
```

```
In [34]: geolocation.head()
```

Out[34]:

	geolocation_zip_code_prefix	geolocation_lat	geolocation_lng	geolocation_city	geolocation_state
0	1037	-23.545621	-46.639292	sao paulo	SP
1	1046	-23.546081	-46.644820	sao paulo	SP
2	1046	-23.546129	-46.642951	sao paulo	SP
3	1041	-23.544392	-46.639499	sao paulo	SP
4	1035	-23.541578	-46.641607	sao paulo	SP

```
In [35]: geolocation = geolocation.drop_duplicates(subset=["geolocation_zip_code_prefix"]).reset_index(drop=True)
```

```
In [36]: geolocation.duplicated().sum()
```

Out[36]: np.int64(0)

```
In [37]: for name, df in datasets.items():
          print("\n", name)
          print(df.dtypes)
```

Customers
customer_id str
customer_unique_id str
customer_zip_code_prefix str
customer_city str
customer_state str
dtype: object

Geolocation
geolocation_zip_code_prefix str
geolocation_lat float64
geolocation_lng float64
geolocation_city str
geolocation_state str
dtype: object

Order Items
order_id str
order_item_id int64
product_id str
seller_id str
shipping_limit_date datetime64[us]
price float64
freight_value float64
dtype: object

Payments
order_id str
payment_sequential int64
payment_type str
payment_installments int64
payment_value float64
dtype: object

Reviews
review_id str
order_id str
review_score int64
review_comment_title str
review_comment_message str
review_creation_date datetime64[us]
review_answer_timestamp datetime64[us]

dtype: object

Orders

order_id	str
customer_id	str
order_status	str
order_purchase_timestamp	datetime64[us]
order_approved_at	datetime64[us]
order_delivered_carrier_date	datetime64[us]
order_delivered_customer_date	datetime64[us]
order_estimated_delivery_date	datetime64[us]
orders_approved_at	datetime64[us]

dtype: object

Products

product_id	str
product_category_name	str
product_name_lenght	float64
product_description_lenght	float64
product_photos_qty	float64
product_weight_g	float64
product_length_cm	float64
product_height_cm	float64
product_width_cm	float64

dtype: object

Sellers

seller_id	str
seller_zip_code_prefix	str
seller_city	str
seller_state	str

dtype: object

Category Translation

product_category_name	str
product_category_name_english	str

dtype: object

EDA (Exploratory Data Analysis)

```
In [38]: order_items.describe()
```

```
Out[38]:
```

	order_item_id	shipping_limit_date	price	freight_value
count	112650.000000	112650	112650.000000	112650.000000
mean	1.197834	2018-01-07 15:36:52.192685	120.653739	19.990320
min	1.000000	2016-09-19 00:15:34	0.850000	0.000000
25%	1.000000	2017-09-20 20:57:27.500000	39.900000	13.080000
50%	1.000000	2018-01-26 13:59:35	74.990000	16.260000
75%	1.000000	2018-05-10 14:34:00.750000	134.900000	21.150000
max	21.000000	2020-04-09 22:35:08	6735.000000	409.680000
std	0.705124	NaN	183.633928	15.806405

```
In [39]: payments.describe()
```

```
Out[39]:
```

	payment_sequential	payment_installments	payment_value
count	103886.000000	103886.000000	103886.000000
mean	1.092679	2.853349	154.100380
std	0.706584	2.687051	217.494064
min	1.000000	0.000000	0.000000
25%	1.000000	1.000000	56.790000
50%	1.000000	1.000000	100.000000
75%	1.000000	4.000000	171.837500
max	29.000000	24.000000	13664.080000

```
In [40]: reviews.describe()
```

Out[40]:

	review_score	review_creation_date	review_answer_timestamp
count	99224.000000	99224	99224
mean	4.086421	2018-01-12 20:49:23.948238	2018-01-16 00:23:56.977939
min	1.000000	2016-10-02 00:00:00	2016-10-07 18:32:28
25%	4.000000	2017-09-23 00:00:00	2017-09-27 01:53:27.250000
50%	5.000000	2018-02-02 00:00:00	2018-02-04 22:41:47.500000
75%	5.000000	2018-05-16 00:00:00	2018-05-20 12:11:21.500000
max	5.000000	2018-08-31 00:00:00	2018-10-29 12:27:35
std	1.347579	NaN	NaN

In [41]: products.describe()

Out[41]:

	product_name_lenght	product_description_lenght	product_photos_qty	product_weight_g	product_length_cm	product_hei
count	32951.000000	32951.000000	32951.000000	32951.000000	32951.000000	32951
mean	48.523656	768.227945	2.166975	2276.376802	30.814725	16
std	10.156155	629.658469	1.728063	4281.926387	16.914005	13
min	5.000000	4.000000	1.000000	0.000000	7.000000	2
25%	42.000000	344.000000	1.000000	300.000000	18.000000	8
50%	51.000000	595.000000	1.000000	700.000000	25.000000	13
75%	57.000000	961.000000	3.000000	1900.000000	38.000000	21
max	76.000000	3992.000000	20.000000	40425.000000	105.000000	105



In [42]: orders["order_status"].value_counts()


```
Out[42]: order_status
delivered      96478
shipped        1107
canceled        625
unavailable     609
invoiced        314
processing      301
created         5
approved        2
Name: count, dtype: int64
```

```
In [43]: payments["payment_type"].value_counts()
```

```
Out[43]: payment_type
credit_card    76795
boleto         19784
voucher        5775
debit_card     1529
not_defined     3
Name: count, dtype: int64
```

```
In [44]: reviews["review_score"].value_counts().sort_index()
```

```
Out[44]: review_score
1      11424
2       3151
3       8179
4      19142
5      57328
Name: count, dtype: int64
```

```
In [45]: customers["customer_state"].value_counts().head(10)
```

```
Out[45]: customer_state
SP      41746
RJ      12852
MG      11635
RS       5466
PR       5045
SC       3637
BA       3380
DF       2140
ES       2033
GO       2020
Name: count, dtype: int64
```

```
In [46]: customers["customer_city"].value_counts().head(10)
```

```
Out[46]: customer_city
sao paulo          15540
rio de janeiro     6882
belo horizonte     2773
brasilia           2131
curitiba           1521
campinas           1444
porto alegre       1379
salvador           1245
guarulhos          1189
sao bernardo do campo  938
Name: count, dtype: int64
```

```
In [47]: sellers["seller_state"].value_counts()
```

```
Out[47]: seller_state
SP      1849
PR       349
MG       244
SC       190
RJ       171
RS       129
GO        40
DF        30
ES        23
BA        19
CE        13
PE         9
PB         6
RN         5
MS         5
MT         4
RO         2
SE         2
AC         1
PI         1
MA         1
AM         1
PA         1
Name: count, dtype: int64
```

```
In [48]: sellers["seller_city"].value_counts().head(10)
```

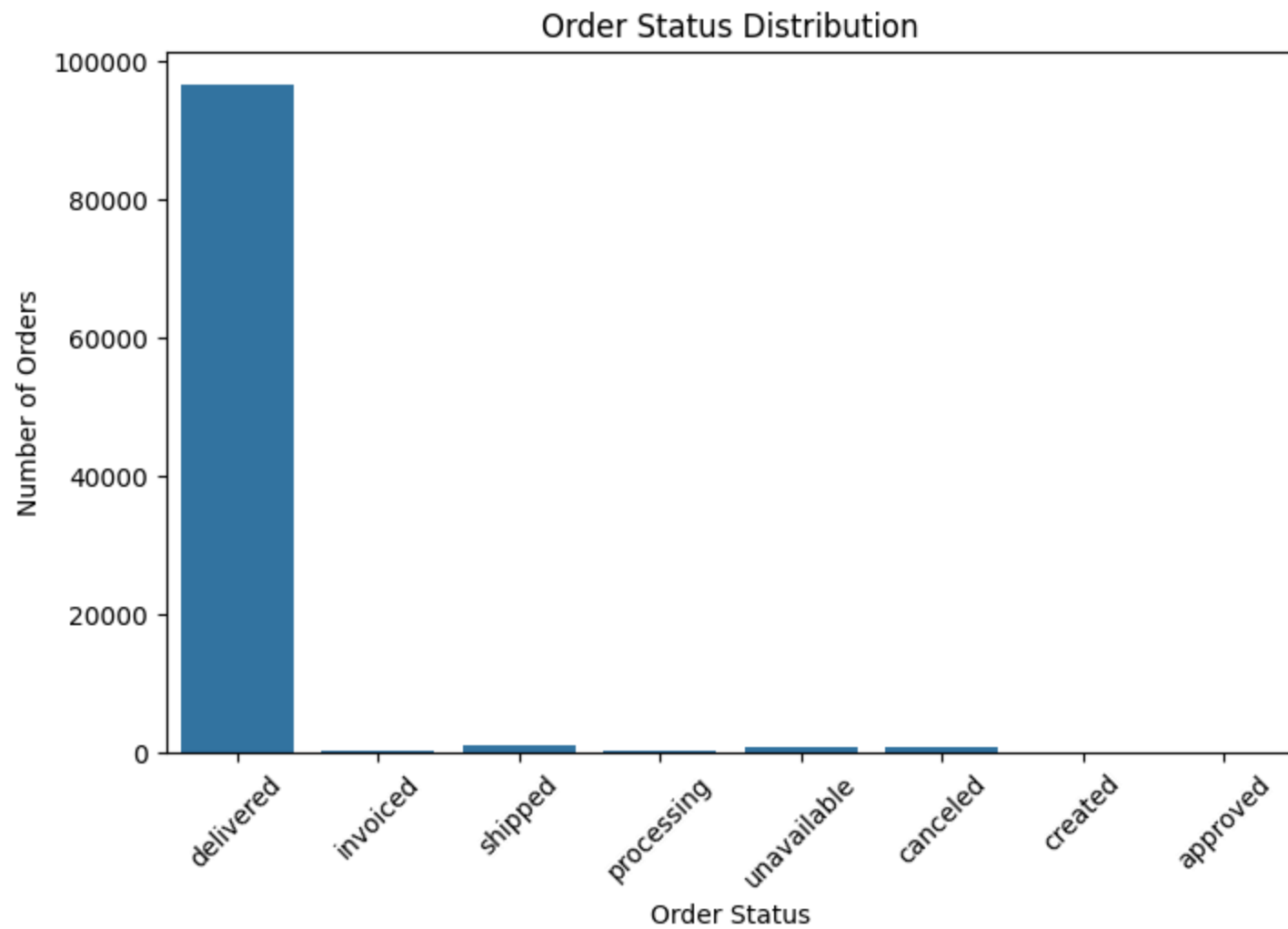
```
Out[48]: seller_city
sao paulo      694
curitiba       127
rio de janeiro  96
belo horizonte  68
ribeirao preto  52
guarulhos      50
ibitinga       49
santo andre    45
campinas       41
maringa        40
Name: count, dtype: int64
```

```
In [49]: products["product_category_name"].value_counts().head(10)
```

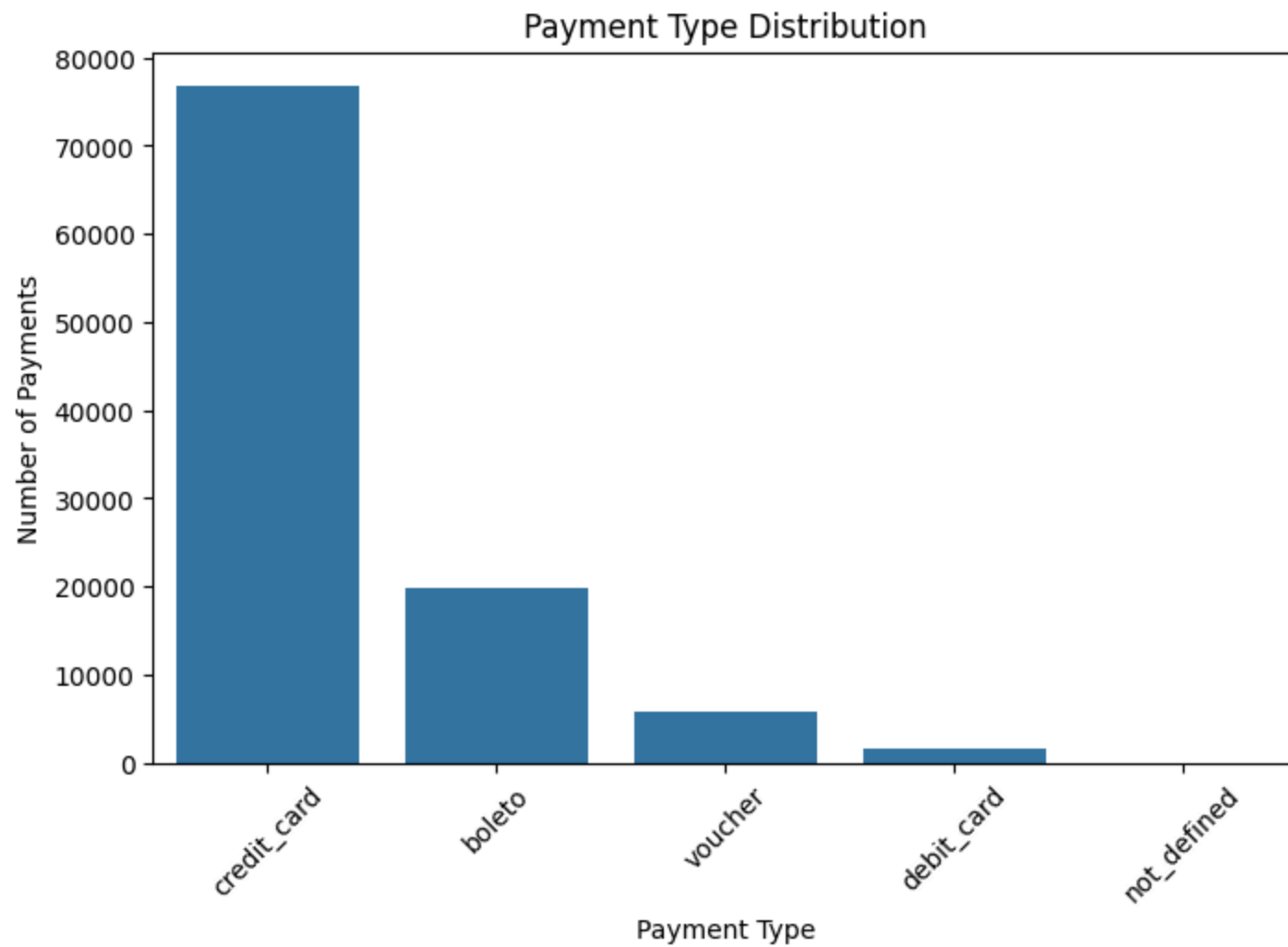
```
Out[49]: product_category_name  
cama_mesa_banho          3029  
esporte_lazer            2867  
moveis_decoracao         2657  
beleza_saude             2444  
utilidades_domesticas    2335  
automotivo               1900  
informatica_acessorios    1639  
brinquedos               1411  
relogios_presentes        1329  
telefonica               1134  
Name: count, dtype: int64
```

Univariate Analysis

```
In [50]: plt.figure(figsize=(8, 5))  
sns.countplot(data=orders, x="order_status")  
plt.xlabel("Order Status")  
plt.ylabel("Number of Orders")  
plt.title("Order Status Distribution")  
plt.xticks(rotation=45)  
plt.show()
```



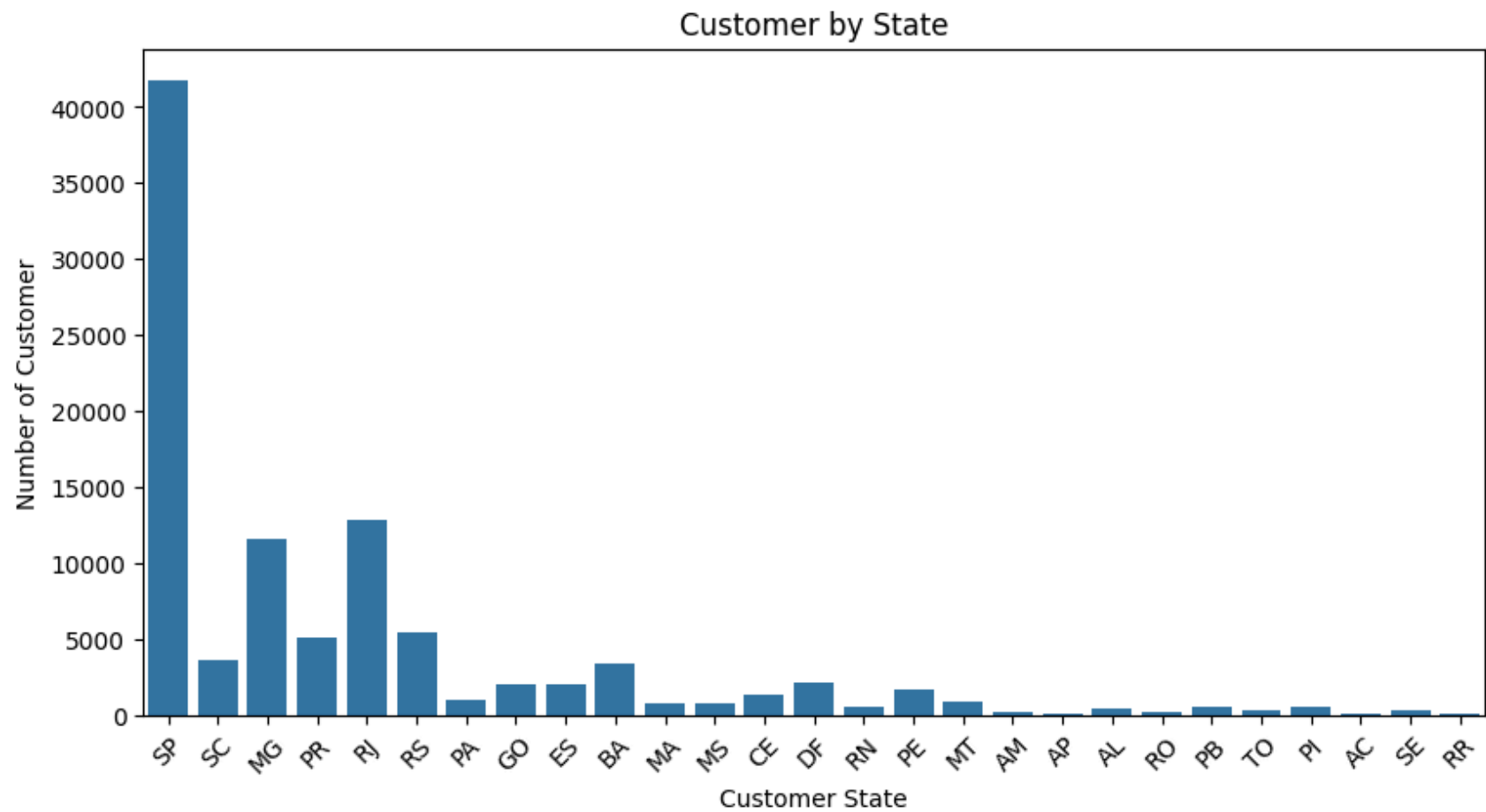
```
In [51]: plt.figure(figsize=(8, 5))
sns.countplot(data=payments, x="payment_type")
plt.xlabel("Payment Type")
plt.ylabel("Number of Payments")
plt.title("Payment Type Distribution")
plt.xticks(rotation=45)
plt.show()
```



```
In [52]: plt.figure(figsize=(8, 5))
sns.countplot(data=reviews, x="review_score")
plt.xlabel("Review Score")
plt.ylabel("Number of Reviews")
plt.title("Review Score Distribution")
plt.xticks(rotation=45)
plt.show()
```



```
In [53]: plt.figure(figsize=(10, 5))
sns.countplot(data=customers,x="customer_state")
plt.xlabel("Customer State")
plt.ylabel("Number of Customer")
plt.title("Customer by State")
plt.xticks(rotation=45)
plt.show()
```



Bivariate Analysis

```
In [54]: plt.figure(figsize=(8,5))
sns.scatterplot(data = order_items,x ="price",y="freight_value")
plt.xlabel("Price")
plt.ylabel("Freight Value")
plt.title("Price vs Freight Value")
plt.show()
```

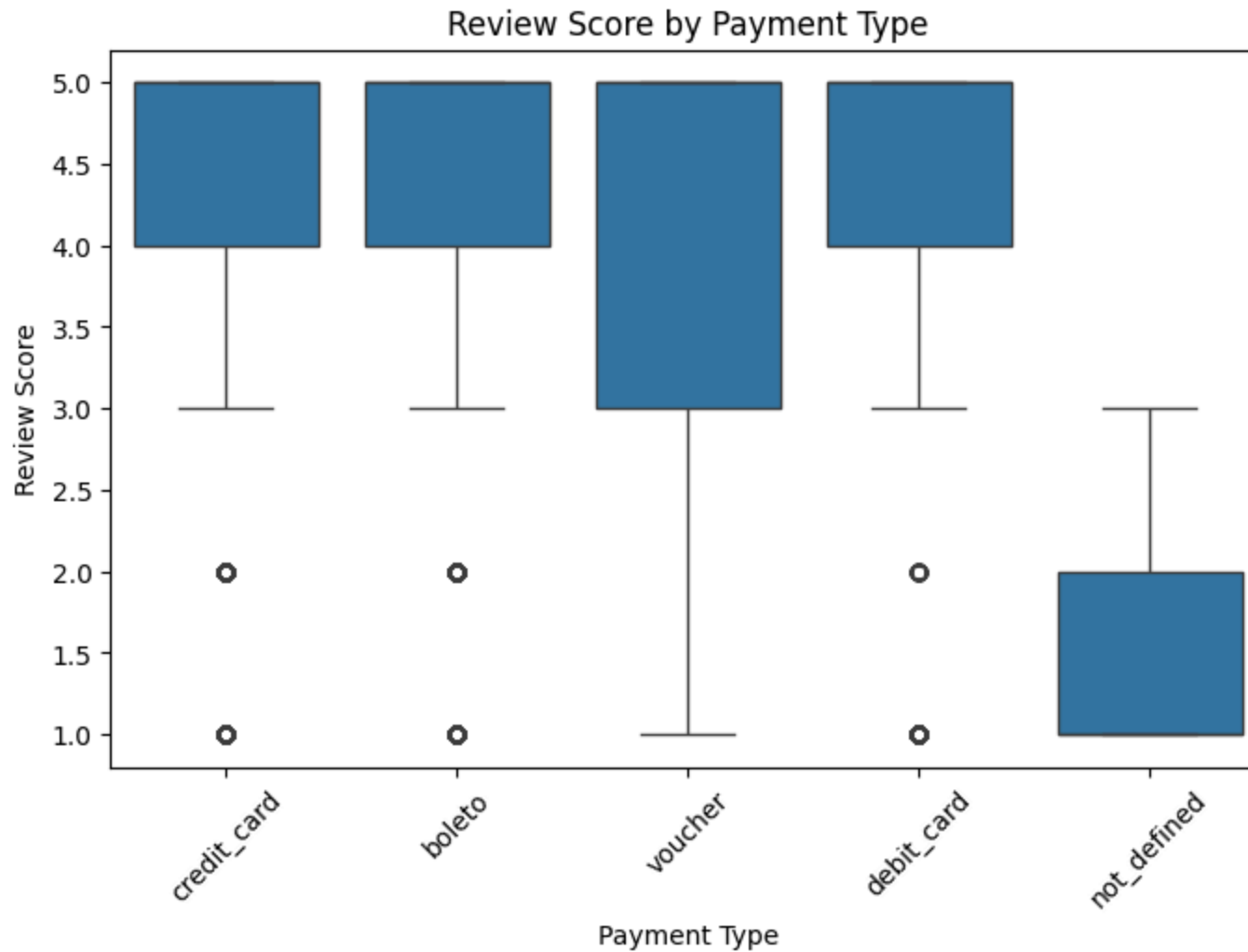



```
In [55]: plt.figure(figsize=(8, 5))

sns.boxplot(
    data=payments.merge(
        reviews[["order_id", "review_score"]],
        on="order_id",
        how="inner"
    ),
    x="payment_type",
    y="review_score"
)

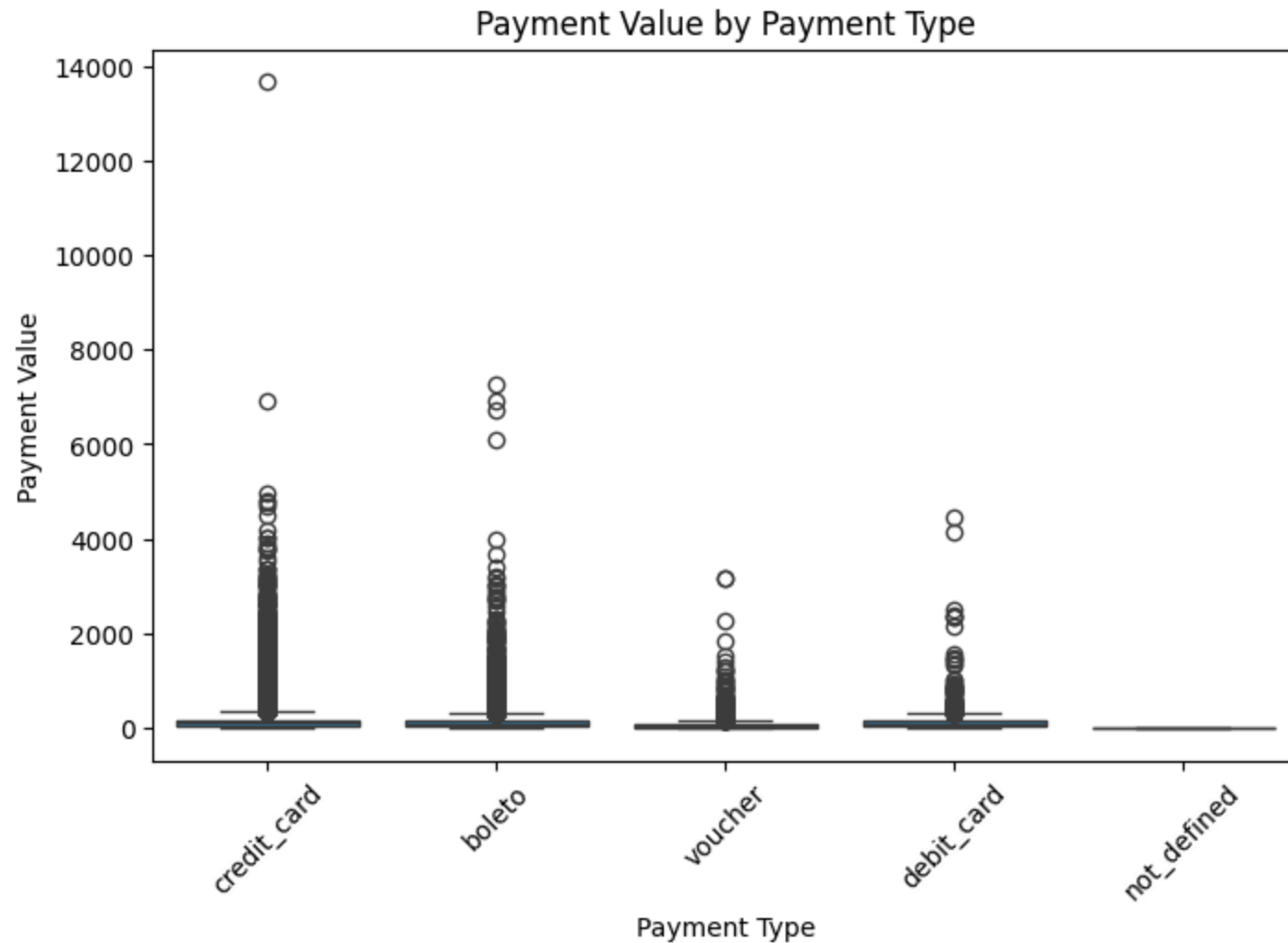
plt.xlabel("Payment Type")
```

```
plt.ylabel("Review Score")
plt.title("Review Score by Payment Type")
plt.xticks(rotation=45)
plt.show()
```



```
In [56]: plt.figure(figsize=(8, 5))
sns.boxplot(data=payments, x="payment_type", y="payment_value")
plt.xlabel("Payment Type")
plt.ylabel("Payment Value")
plt.title("Payment Value by Payment Type")
```

```
plt.xticks(rotation=45)
plt.show()
```



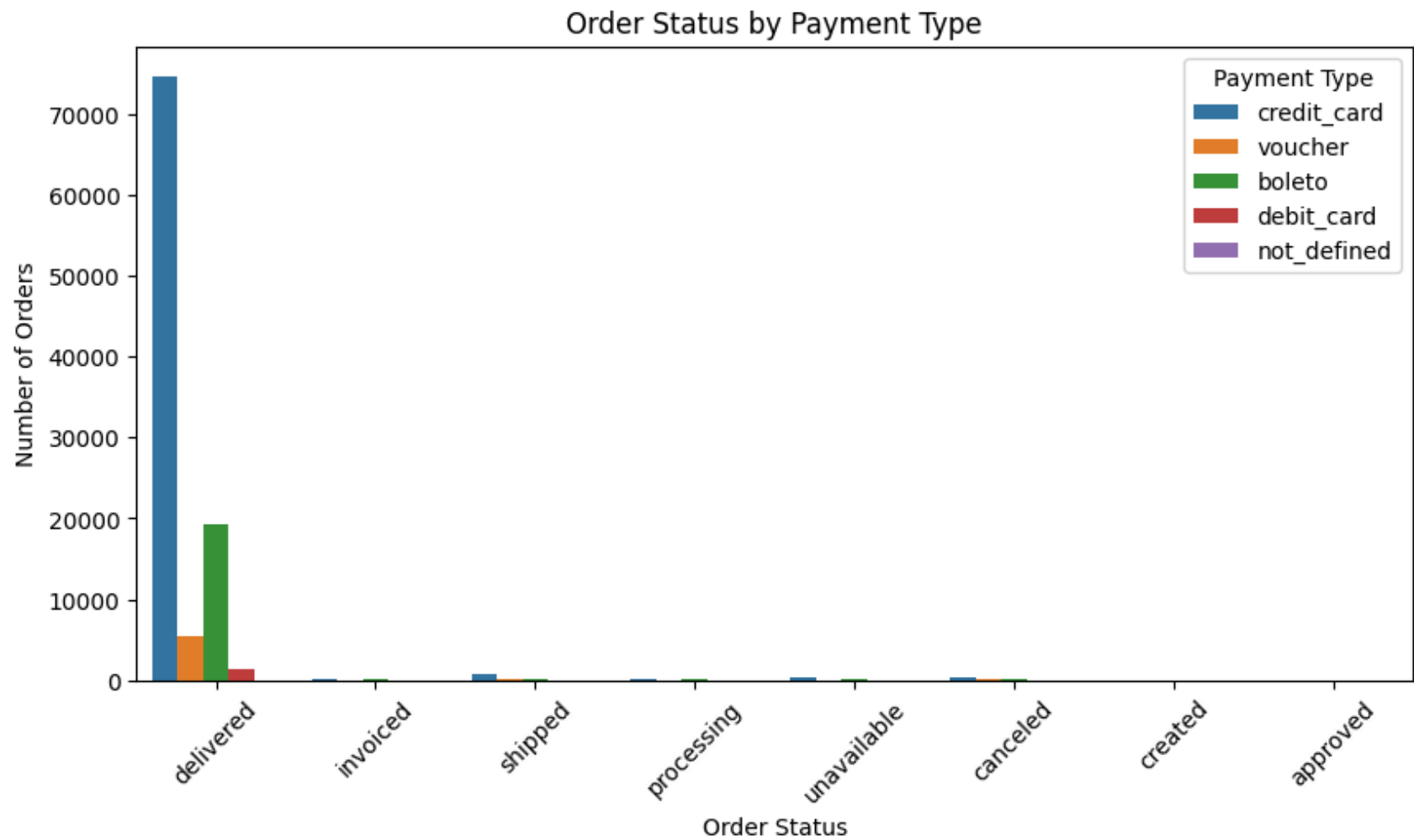
```
In [57]: orders["delivery_time"] = (orders["order_delivered_customer_date"] - orders["order_purchase_timestamp"]).dt.days
delivery_review = orders.merge(reviews[["order_id", "review_score"]], on="order_id", how="inner")
plt.figure(figsize=(8, 5))
sns.scatterplot(
    data=delivery_review,
    x="delivery_time",
    y="review_score"
```

```
)
plt.xlabel("Delivery Time (Days)")
plt.ylabel("Review Score")
plt.title("Delivery Time vs Review Score")
plt.show()
```



```
In [58]: order_payment = orders.merge(
    payments[["order_id", "payment_type"]],
    on="order_id",
    how="inner"
)
plt.figure(figsize=(10, 5))
```

```
sns.countplot(  
    data=order_payment,  
    x="order_status",  
    hue="payment_type"  
)  
  
plt.xlabel("Order Status")  
plt.ylabel("Number of Orders")  
plt.title("Order Status by Payment Type")  
plt.xticks(rotation=45)  
plt.legend(title="Payment Type")  
plt.show()
```



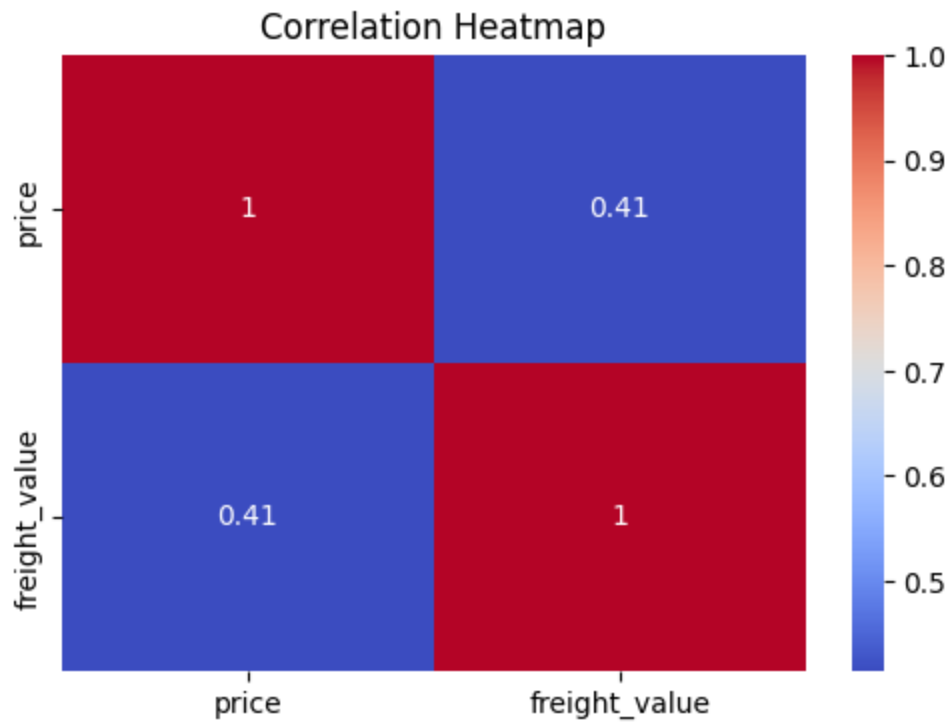
Correlation Analysis

```
In [59]: numeric_date = order_items[["price", "freight_value"]]  
numeric_date.head()  
correlation = numeric_date.corr()  
correlation
```

Out[59]:

	price	freight_value
price	1.000000	0.414204
freight_value	0.414204	1.000000

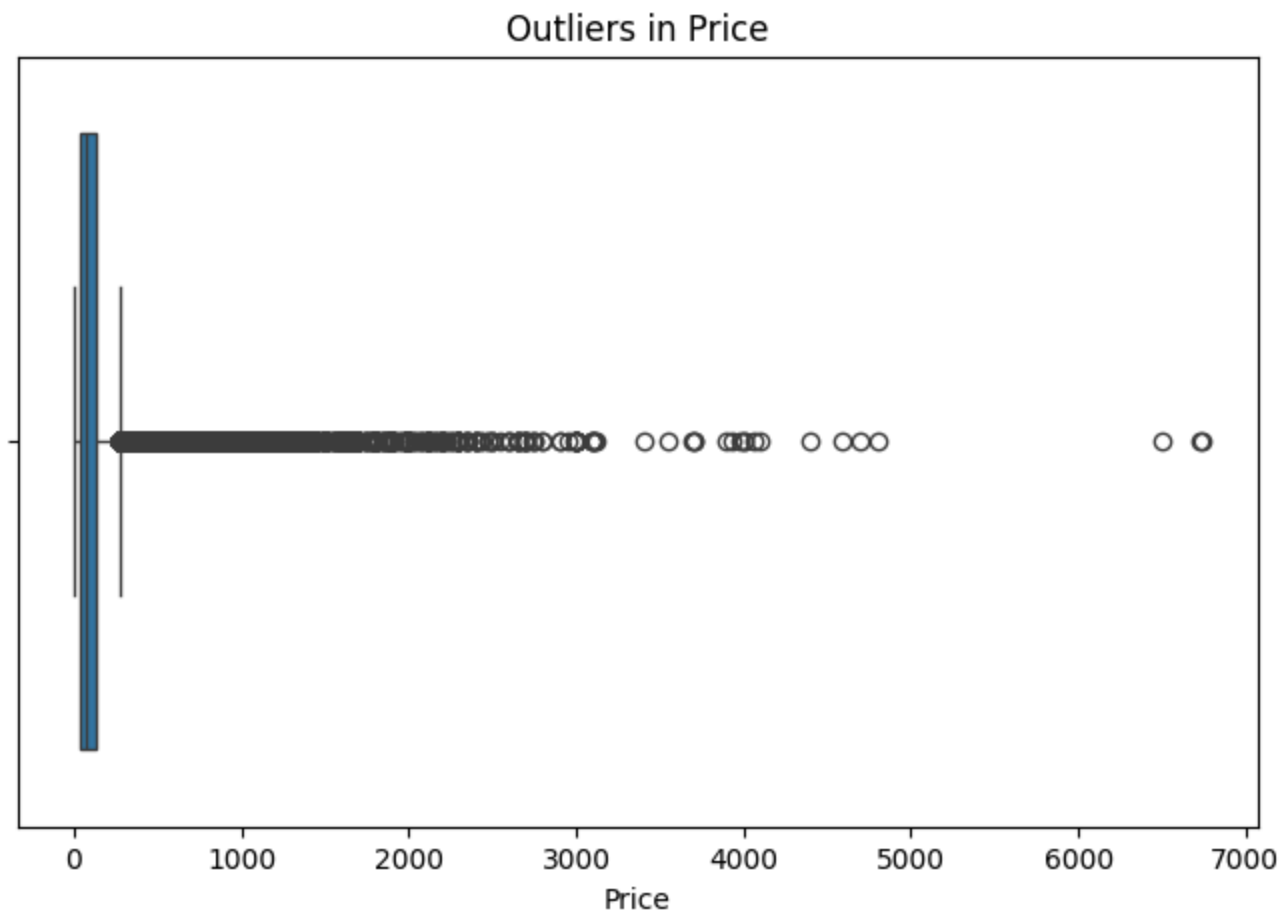
```
In [60]: plt.figure(figsize=(6,4))
sns.heatmap(correlation,annot=True,cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```



```
In [61]: plt.figure(figsize=(8, 5))

sns.boxplot(x=order_items["price"])

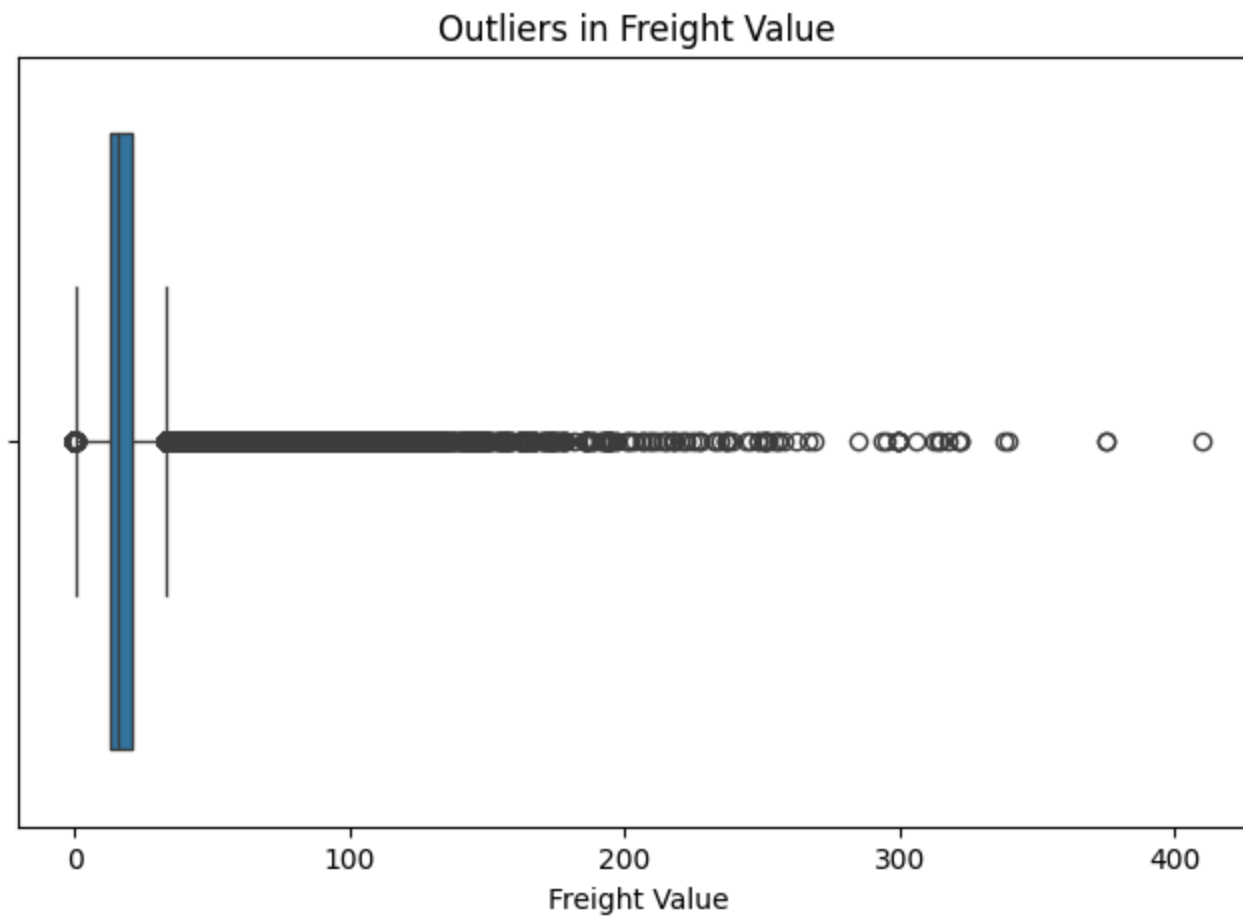
plt.title("Outliers in Price")
plt.xlabel("Price")
plt.show()
```



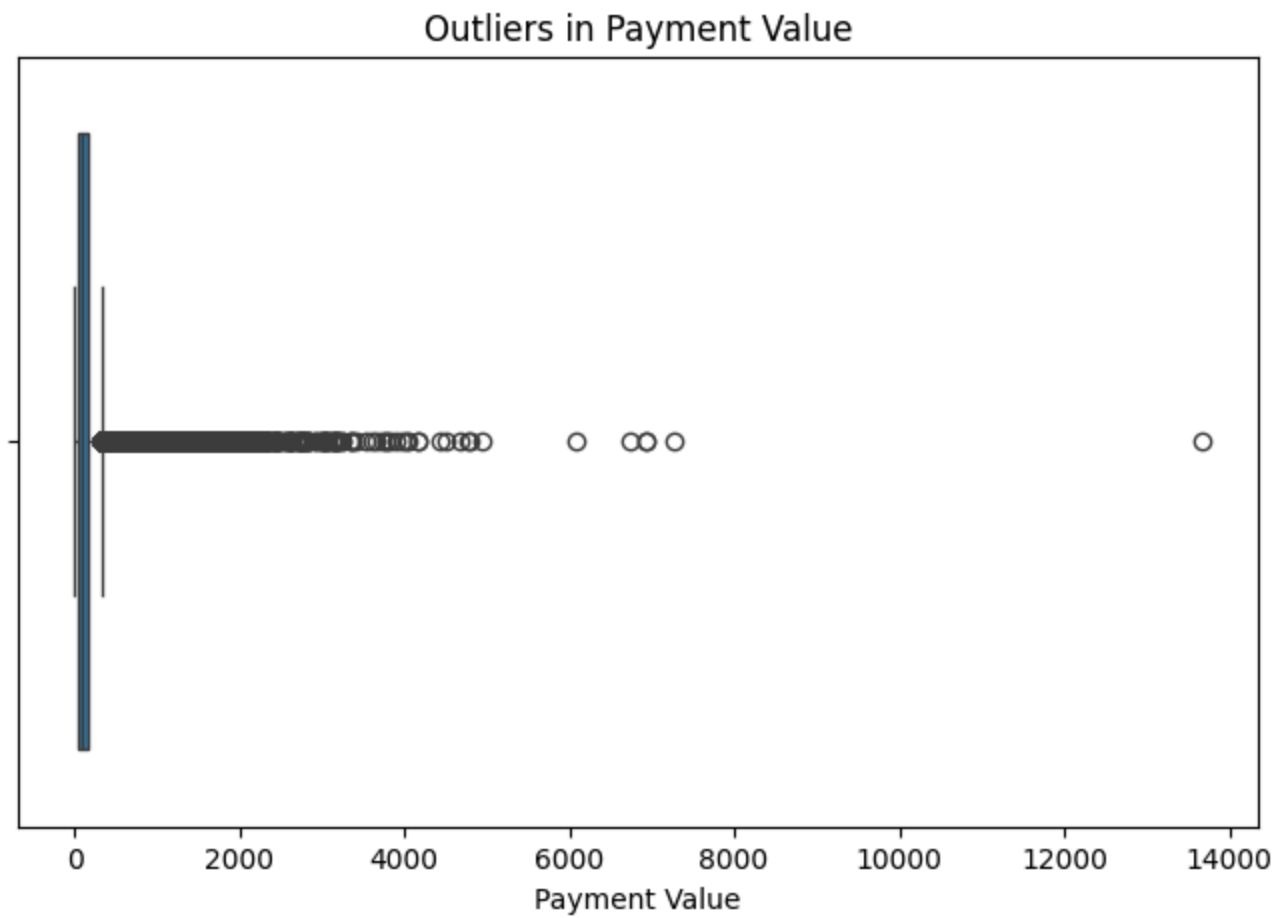
```
In [62]: plt.figure(figsize=(8, 5))

sns.boxplot(x=order_items["freight_value"])

plt.title("Outliers in Freight Value")
plt.xlabel("Freight Value")
plt.show()
```

```
In [63]: plt.figure(figsize=(8, 5))
sns.boxplot(x=payments["payment_value"])
plt.title("Outliers in Payment Value")
plt.xlabel("Payment Value")
plt.show()
```



```
In [65]: Q1 = order_items["price"].quantile(0.25)
Q3 = order_items["price"].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

price_outliers = order_items[(order_items["price"] < lower) | (order_items["price"] > upper)]

print("Number of price outliers:", len(price_outliers))
```

Number of price outliers: 8427

```
In [67]: Q1 = order_items["freight_value"].quantile(0.25)
Q3 = order_items["freight_value"].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

freight_outliers = order_items[
    (order_items["freight_value"] < lower) |
    (order_items["freight_value"] > upper)
]

print("Number of freight value outliers:", len(freight_outliers))
```

Number of freight value outliers: 12134

```
In [70]: Q1 = payments["payment_value"].quantile(0.25)
Q3 = payments["payment_value"].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

payment_outliers = payments[
    (payments["payment_value"] < lower) |
    (payments["payment_value"] > upper)
]

print("Number of payment value outliers:", len(payment_outliers))
```

Number of payment value outliers: 7981

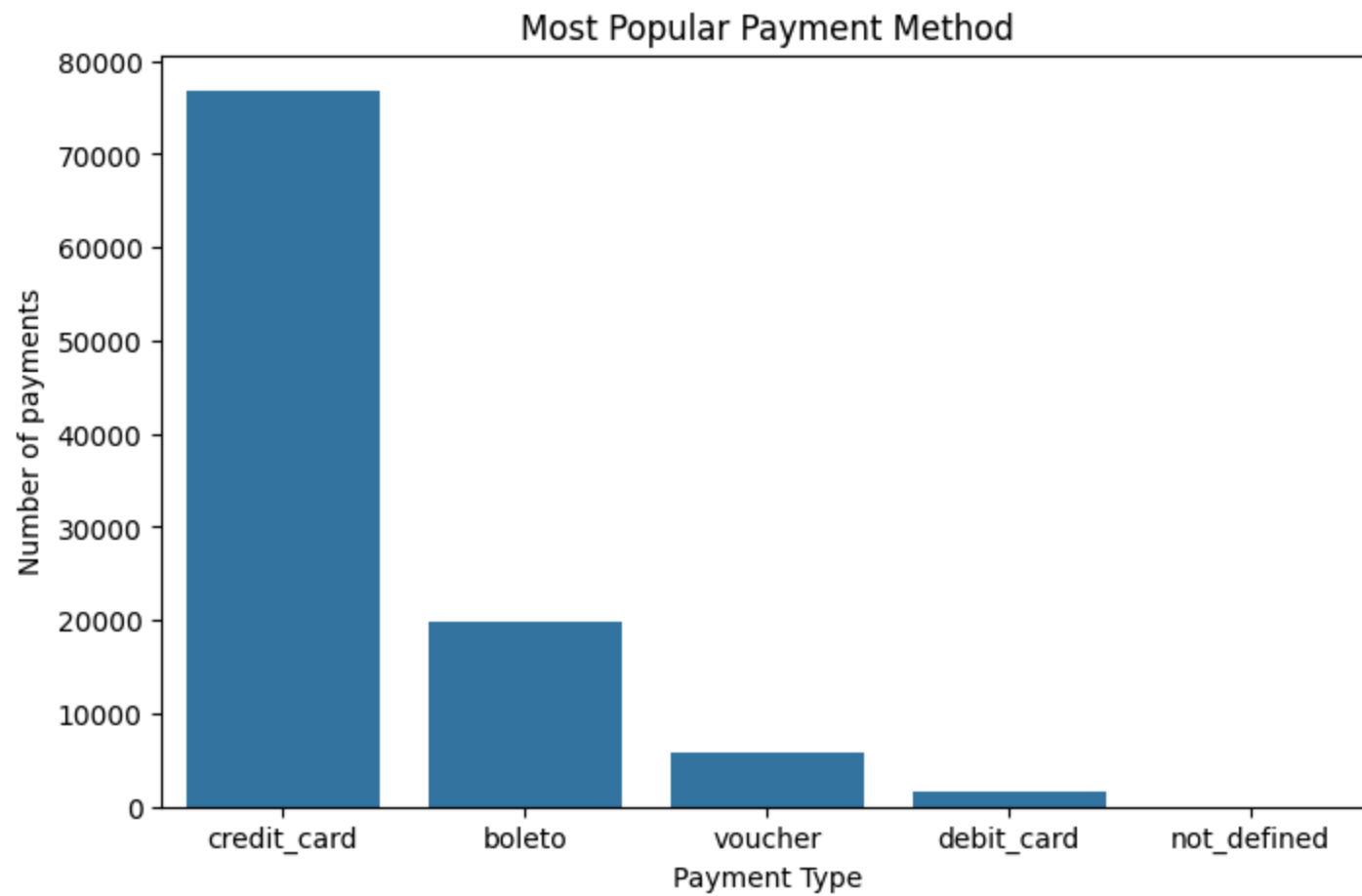
Business Question

```
In [72]: # 1.Which payment method is most popular?
```

```
In [76]: payment_counts = payments["payment_type"].value_counts()
print(payment_counts)
```

```
plt.figure(figsize=(8,5))
sns.barplot(
    x = payment_counts.index,
    y = payment_counts.values
)
plt.xlabel("Payment Type")
plt.ylabel("Number of payments")
plt.title("Most Popular Payment Method")
plt.show()
```

```
payment_type
credit_card    76795
boleto         19784
voucher         5775
debit_card     1529
not_defined      3
Name: count, dtype: int64
```

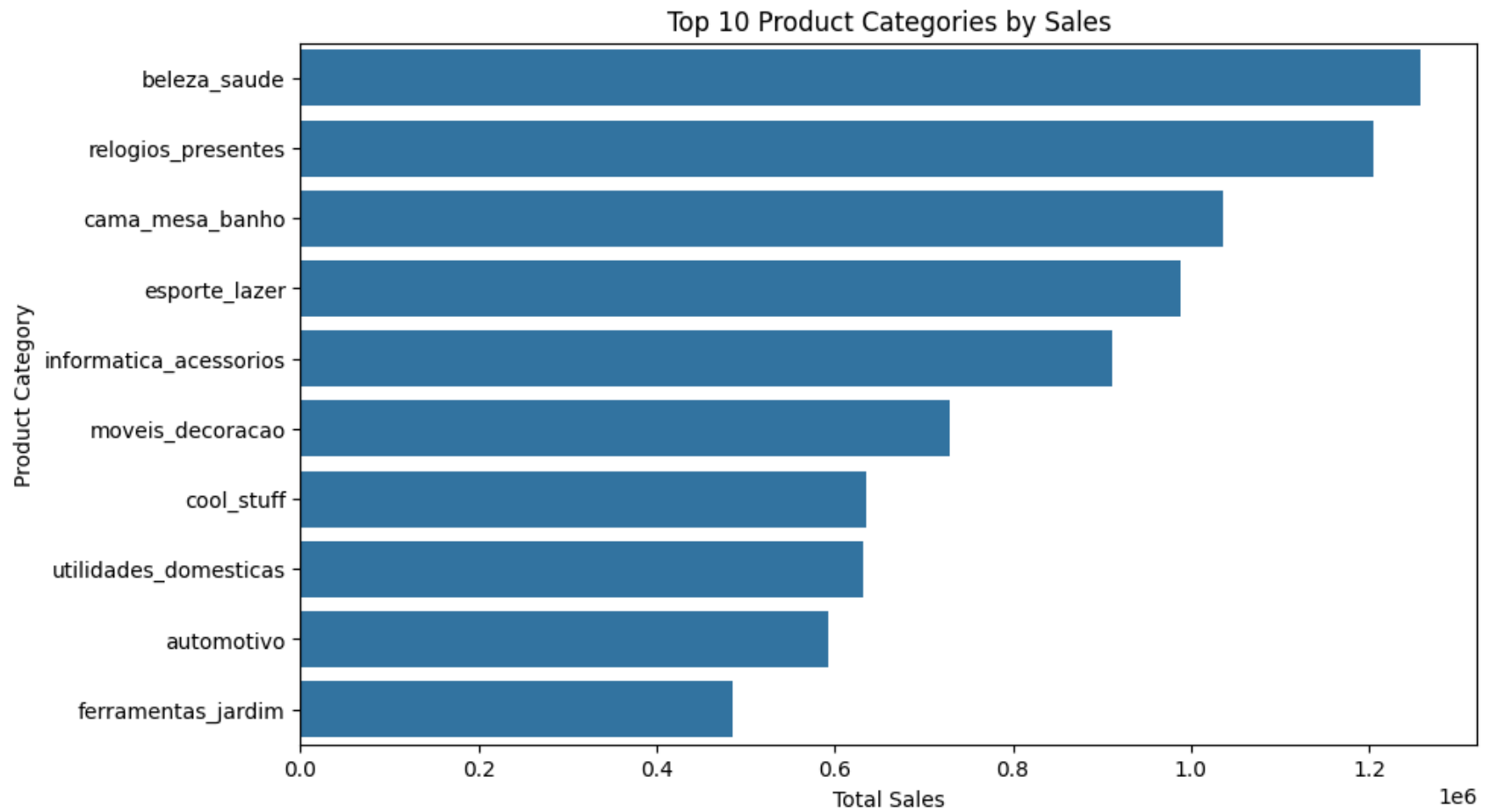


In [77]: *#2.Which product categories generate the most sales?*

```
In [80]: product_sales = order_items.merge(products[["product_id","product_category_name"]], on = "product_id",how="inner")
category_sales = (product_sales.groupby("product_category_name")["price"].sum().sort_values(ascending=False))
category_sales.head(10)
```

```
Out[80]: product_category_name
        beleza_saude          1258681.34
        relorios_presentes    1205005.68
        cama_mesa_banho       1036988.68
        esporte_lazer         988048.97
        informatica_acessorios 911954.32
        moveis_decoracao       729762.49
        cool_stuff             635290.85
        utilidades_domesticas  632248.66
        automotivo             592720.11
        ferramentas_jardim     485256.46
        Name: price, dtype: float64
```

```
In [95]: plt.figure(figsize=(10, 6))
        sns.barplot(
            x=category_sales.head(10).values,
            y=category_sales.head(10).index)
        plt.xlabel("Total Sales")
        plt.ylabel("Product Category")
        plt.title("Top 10 Product Categories by Sales")
        plt.show()
```



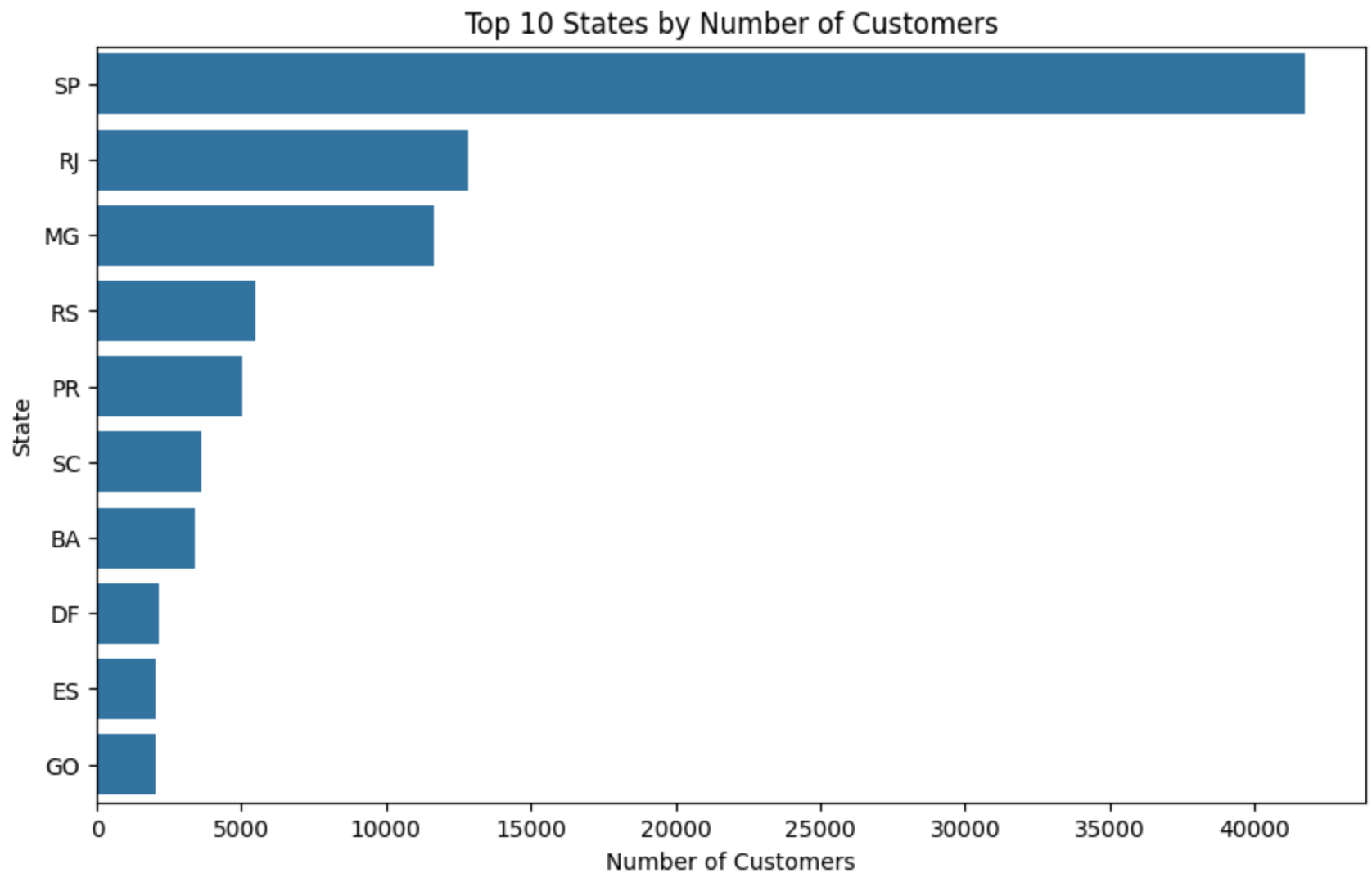
In [82]: *#3 Which states have the highest number of customers?*

```
In [83]: customer_state = customers["customer_state"].value_counts()  
customer_state.head(10)
```

```
Out[83]: customer_state
SP      41746
RJ      12852
MG      11635
RS       5466
PR       5045
SC       3637
BA       3380
DF       2140
ES       2033
GO       2020
Name: count, dtype: int64
```

```
In [96]: plt.figure(figsize=(10, 6))
sns.barplot(
    x=customer_state.head(10).values,
    y=customer_state.head(10).index)

plt.xlabel("Number of Customers")
plt.ylabel("State")
plt.title("Top 10 States by Number of Customers")
plt.show()
```

In [84]: *#4 What is the monthly sales trend*

```
In [85]: sales_data = order_items.merge(orders[["order_id", "order_purchase_timestamp"]], on="order_id", how="inner")

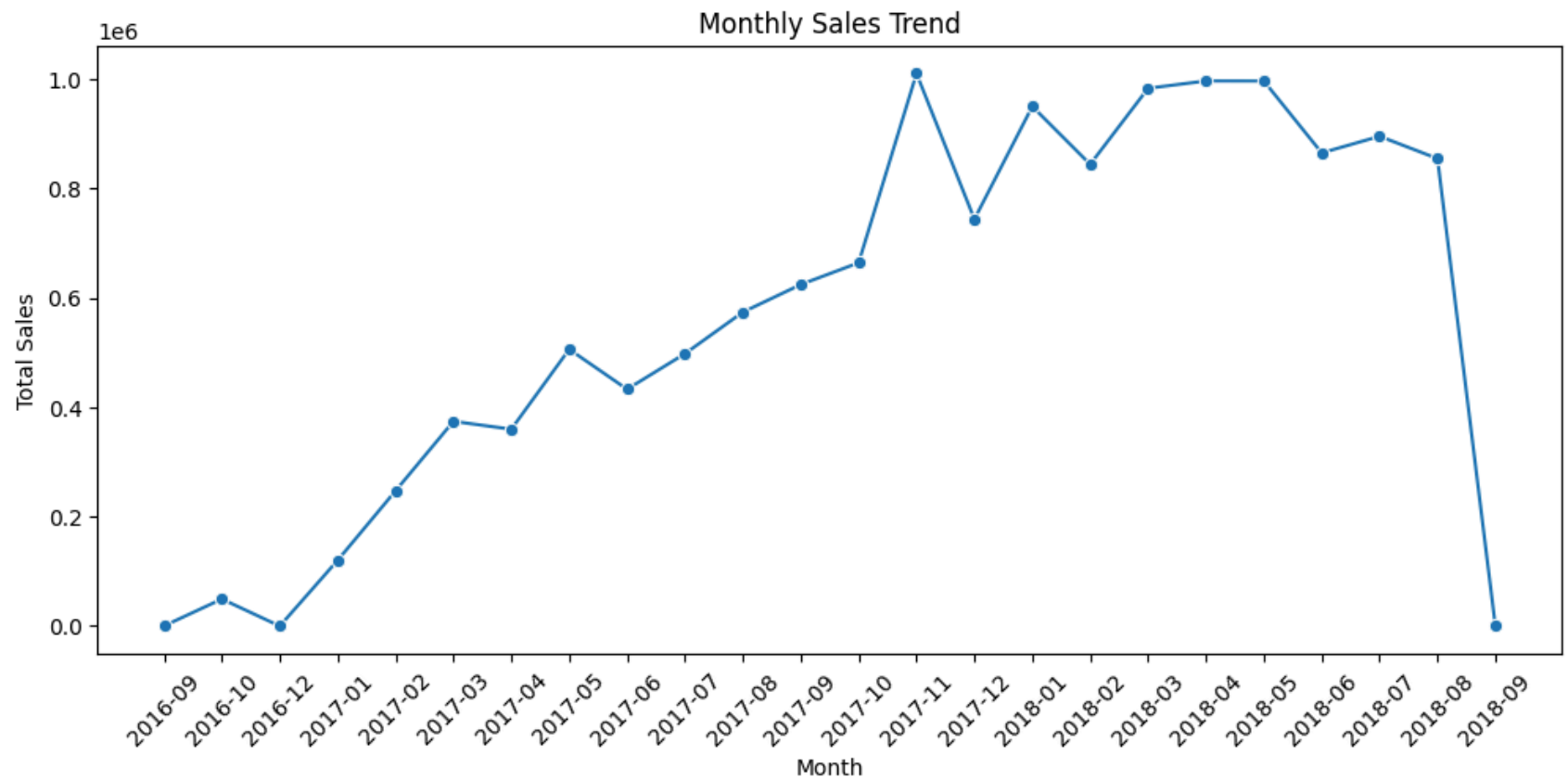
sales_data["order_purchase_timestamp"] = pd.to_datetime(sales_data["order_purchase_timestamp"])
sales_data["month"] = sales_data["order_purchase_timestamp"].dt.to_period("M")
```

```
monthly_sales = (sales_data.groupby("month")["price"].sum())
monthly_sales
```

```
Out[85]: month
2016-09      267.36
2016-10    49507.66
2016-12      10.90
2017-01    120312.87
2017-02    247303.02
2017-03    374344.30
2017-04    359927.23
2017-05    506071.14
2017-06    433038.60
2017-07    498031.48
2017-08    573971.68
2017-09    624401.69
2017-10    664219.43
2017-11    1010271.37
2017-12    743914.17
2018-01    950030.36
2018-02    844178.71
2018-03    983213.44
2018-04    996647.75
2018-05    996517.68
2018-06    865124.31
2018-07    895507.22
2018-08    854686.33
2018-09      145.00
Freq: M, Name: price, dtype: float64
```

```
In [97]: plt.figure(figsize=(12, 5))
sns.lineplot(
    x=monthly_sales.index.astype(str),
    y=monthly_sales.values,
    marker="o")

plt.xlabel("Month")
plt.ylabel("Total Sales")
plt.title("Monthly Sales Trend")
plt.xticks(rotation=45)
plt.show()
```



In [86]: #5 Which sellers generate the highest revenue?

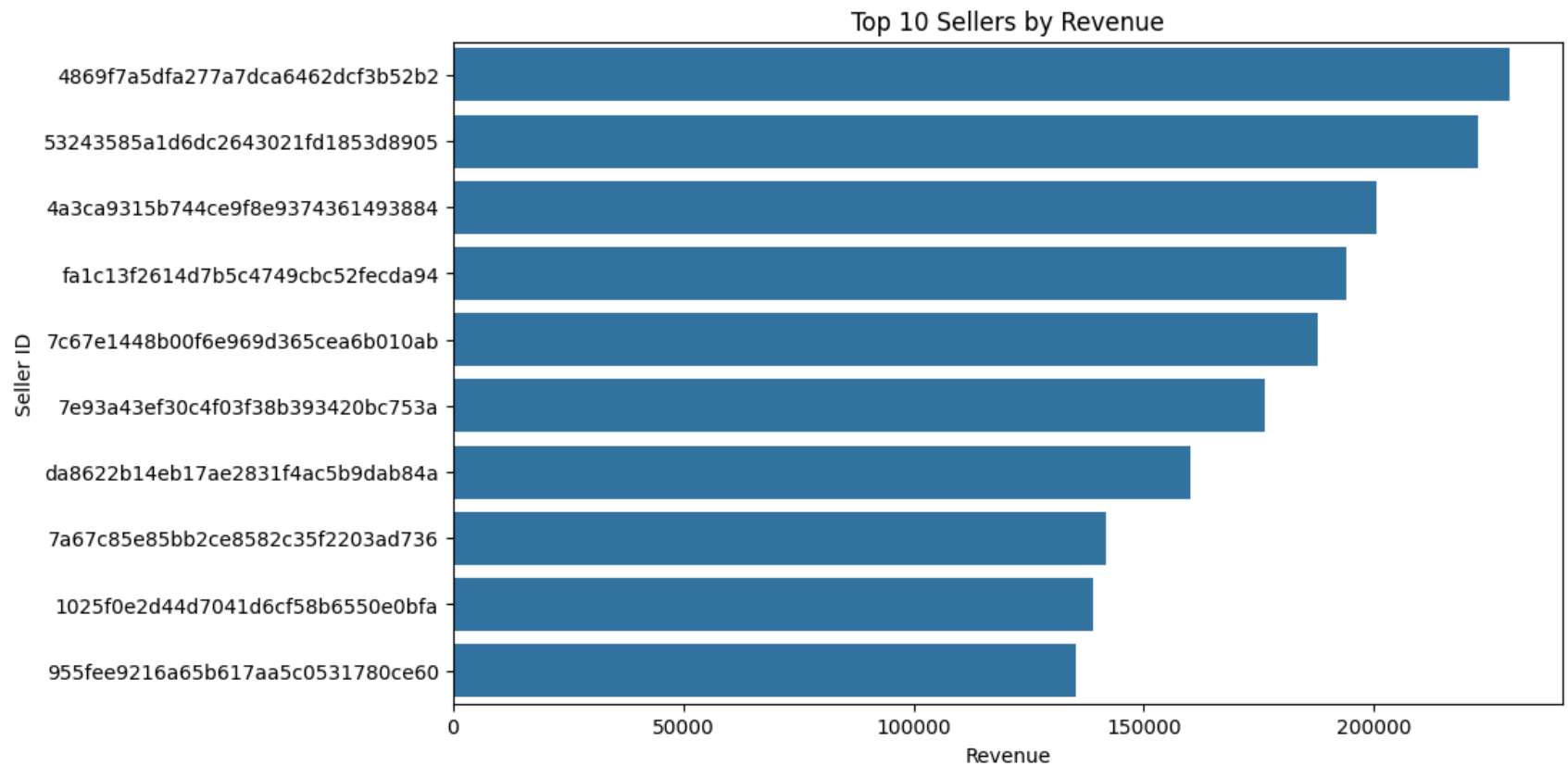
```
In [87]: seller_sales = (order_items.groupby("seller_id")["price"].sum().sort_values(ascending=False))
seller_sales.head(10)
```

```
Out[87]: seller_id
4869f7a5dfa277a7dca6462dcf3b52b2    229472.63
53243585a1d6dc2643021fd1853d8905    222776.05
4a3ca9315b744ce9f8e9374361493884    200472.92
fa1c13f2614d7b5c4749cbc52fecda94    194042.03
7c67e1448b00f6e969d365cea6b010ab    187923.89
7e93a43ef30c4f03f38b393420bc753a    176431.87
da8622b14eb17ae2831f4ac5b9dab84a    160236.57
7a67c85e85bb2ce8582c35f2203ad736    141745.53
1025f0e2d44d7041d6cf58b6550e0bfa    138968.55
955fee9216a65b617aa5c0531780ce60    135171.70
Name: price, dtype: float64
```

```
In [98]: plt.figure(figsize=(10, 6))

sns.barplot(
    x=seller_sales.head(10).values,
    y=seller_sales.head(10).index)

plt.xlabel("Revenue")
plt.ylabel("Seller ID")
plt.title("Top 10 Sellers by Revenue")
plt.show()
```



In [88]: #6. What is the average delivery time?

```
In [90]: orders["delivery_time"] = (orders["order_delivered_customer_date"] - orders["order_purchase_timestamp"]).dt.days
average_delivery_time = orders["delivery_time"].mean()
print("Average Delivery Time:", average_delivery_time, "days")
```

Average Delivery Time: 12.094085575687217 days

In [91]: #7. Which product categories have the highest average review scores?

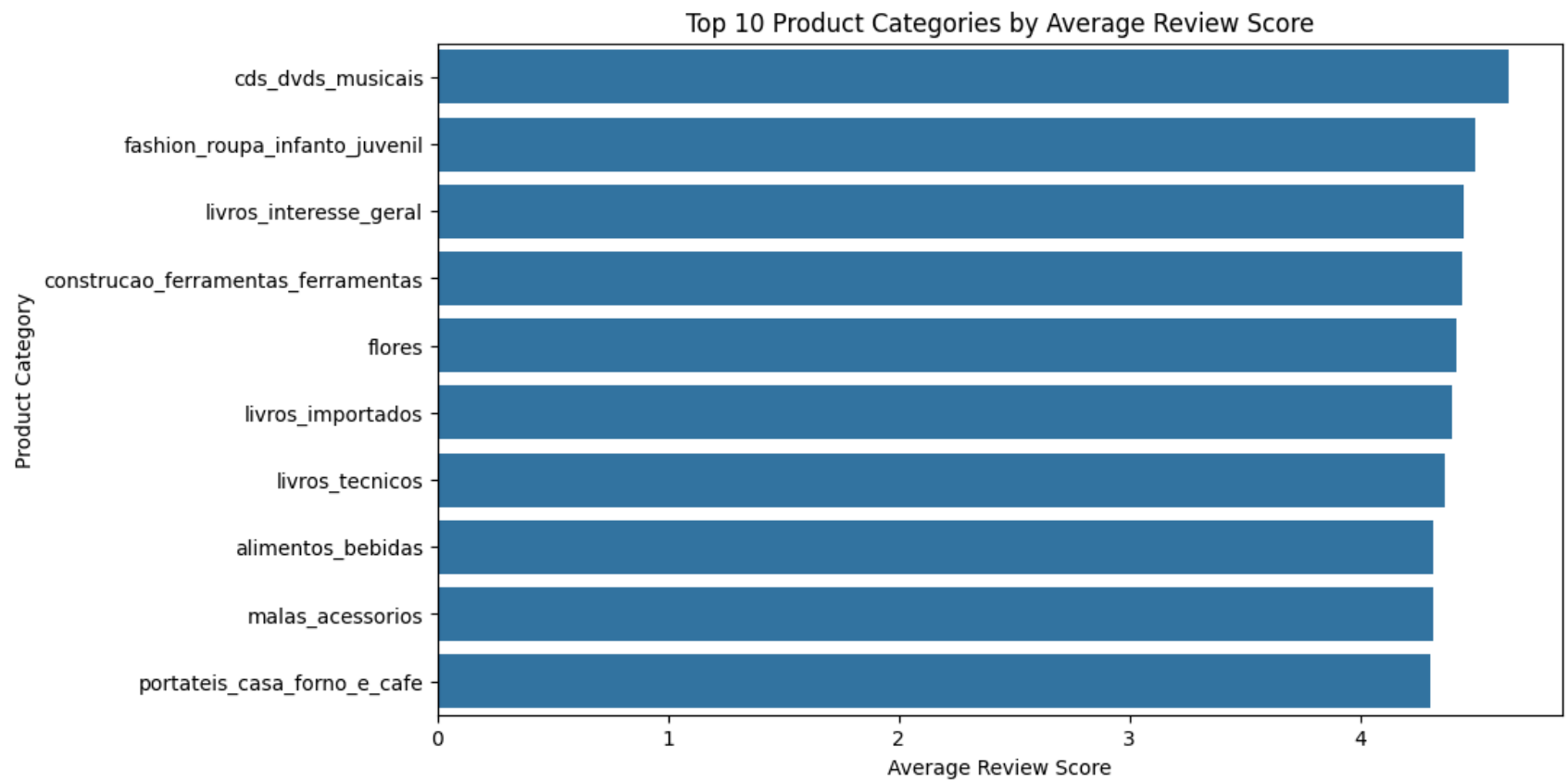
```
In [94]: category_reviews = order_items.merge(products[["product_id", "product_category_name", ]], on = "product_id", how = "inner")
category_reviews = category_reviews.merge(reviews[["order_id", "review_score"]], on = "order_id", how = "inner")
category_review_score = (category_reviews.groupby("product_category_name")["review_score"].mean().sort_values(ascending=False).head(10))
```

```
Out[94]: product_category_name
          cds_dvds_musicais      4.642857
          fashion_roupa_infanto_juvenil  4.500000
          livros_interesse_geral      4.446266
          construcao_ferramentas_ferramentas  4.444444
          flores      4.419355
          livros_importados      4.400000
          livros_tecnicos      4.368421
          alimentos_bebidas      4.315412
          malas_acessorios      4.315257
          portateis_casa_forno_e_cafe      4.302632
          Name: review_score, dtype: float64
```

```
In [99]: plt.figure(figsize=(10, 6))

sns.barplot(
    x=category_review_score.head(10).values,
    y=category_review_score.head(10).index
)

plt.xlabel("Average Review Score")
plt.ylabel("Product Category")
plt.title("Top 10 Product Categories by Average Review Score")
plt.show()
```



In []: